

УТВЕРЖДАЮ

Заместитель директора Новосибирского
филиала – директор по прикладным проектам
ПАО «Ростелеком»

_____ П.А. Ташлыкова

«_____» _____ 2024

МП

УТВЕРЖДАЮ

Ректор Новосибирского государственного
университета

_____ М.П. Федорук

«_____» _____ 2024

МП

СОГЛАСОВАНО

И.о. директора ЦИИ НГУ

_____ А.В. Матвеев

«_____» _____ 2024

ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ

Договор с Индустриальным партнёром на выполнение работ или оказание услуг в целях совместной реализации
ключевых мероприятий программы Центра в части внебюджетного софинансирования от 23.11.2023 г. №
О-35-2023 по пунктам 7-12 Плана Работ к техническому заданию №2 «Разработка программного комплекса для
детектирования событий по тематике городского транспорта и дорожного движения»

Руководитель разработки,
ведущий научный сотрудник ЦИИНГУ

_____ А.Г. Окунев

«_____» _____ 2024

СОДЕРЖАНИЕ:

1. Объект испытаний	3
2. Цель испытаний	4
3. Требования к программе	4
4. Требования к программной документации	5
5. Средства и порядок испытаний	6
6. Методы испытаний системы детектирования событий	7
7. Результаты предварительных испытаний	11
8. Сценарии для испытаний детекторов	12

1. Объект испытаний

Испытаниям подвергается Система детектирования событий в системах городского транспорта и управления дорожным движением.

Далее под Системой детектирования событий в системах городского транспорта и управления дорожным движением понимается программное обеспечение для обработки видеопотока и архивных видеоданных для идентификации определённых типов объектов и событий с помощью предварительно настроенных детекторов, разработанных на основе алгоритмов машинного обучения.

Данные детекторы предназначены для обработки медиа контента:

- видеопотока СВН в реальном времени;
- архивного видеопотока СВН;
- статических фотоизображений, являющихся скриншотами видеопотока СВН.

Испытаниям подвергаются следующие детекторы:

- Детектор марки, модели и цвета ТС.
- Детектор ДТП на перекрестке.
- Детектор выпавшего груза на перекрестке.
- Детектор остановки в полосе движения.
- Детектор пешехода.
- Детектор непристегнутого ремня / разговора по телефону.

2. Цель испытаний

Цель проведения испытаний – проверка соответствия характеристик разработанных детекторов общим и системным требованиям, изложенным в документе “Техническое задание”.

3. Требования к программе

Задача оператора испытаний – убедиться, что при разработке детекторов учтены нижеперечисленные требования:

- Осуществлена разметка обучающих и тестовых изображений (датасетов).
- Программное обеспечение разработано с учетом совместимости с российскими операционными системами. Программное обеспечение использовало открытые фреймворки машинного обучения (PyTorch), ускорения инференса моделей (ONNX Runtime) и контейнеризации приложений (Docker)
- Подготовленные датасеты обеспечивают воспроизводимые тренировки и валидацию моделей, а также тестирование количественных параметров представляемых моделей.
- Обеспечена возможность интеграции каждой из разработанных моделей детектирования с Платформой АР. Время передачи результатов потребителям через API с момента вычисления результата обработки должно быть не более 5 минут.

4. Требования к программной документации

Для проведения испытаний в качестве результатов работ для каждого из разработанных детекторов должны быть предоставлены:

- исходный программный код и конфигурационные настройки разработанной модели;
- размеченные входной, тренировочный датасеты, использованные для разработки и проверки модели;
- описание программы, описание модели и краткое описание математических алгоритмов, задействованных для построения модели;
- системные характеристики оборудования, задействованного для разработки и тренировки модели.
- руководство оператора, программиста и системного программиста.

5. Средства и порядок испытаний

Предварительные испытания проводятся на графическом кластере ВКИ НГУ с использованием графических ускорителей NVIDIA Tesla V100.

Приемо-сдаточные испытания проводятся на сервере виртуальных машин ВКИ с конфигурацией 8 ядер CPU, 16Гб ОЗУ, а также на одноплатных компьютерах NVIDIA Jetson NX Xavier 8Gb с операционной системой JetPack 5.

Программные средства для каждого детектора должны быть оформлены в виде отдельного репозитория. Репозиторий включает размеченные тренировочный, валидационный (тестовый при необходимости) датасеты, скрипты для тренировки моделей, скрипты для вычисления метрик моделей на валидационном и/или тестовом датасете, скрипты для демонстрационного инференса моделей на RTSP потоках заказчика в течение 15-секундного интервала.

В процессе инференса должны быть получены:

- результаты в формате, определенные Техническим заданием;
- демонстрационный видеоролик с отображением детектируемых объектов и ключевых параметров.

Для демонстрационных моделей, подключенных к веб-интерфейсу вывод результатов должен осуществляться с использованием указанных интерфейсов. Для неподключенных моделей вывод результатов осуществляется в папку results. Название json файла и видео файла должны быть в формате «timestamp.json» и «timestamp.mp4», где timestamp — результат метода time.time() в формате int, соответствующий началу обработки RTSP потока.

Испытания проводятся с использованием средства контейнеризации Docker. Программные пакеты и модули должны быть собраны в Docker контейнере. При работе с типовым Docker контейнером должен быть предъявлен скрипт для автоматической установки необходимых зависимостей.

Системные программные средства, используемые программой: операционная система Ubuntu версии 22+ и JetPack 5.

6. Методы испытаний системы детектирования событий

Для проведения испытаний моделей оператор должен проделать следующие действия:

Проверить выполнение предусловий:

1. Доступ к серверу ВКИ НГУ.
2. Наличие программного обеспечения, совместимого с российскими операционными системами, и инструментария машинного обучения, такого как PyTorch и ONNX Runtime, а также TensorRT для Jetson.
3. Подготовленные репозитории для каждого детектора, включающие:
 - Размеченные тренировочные, валидационные и тестовые датасеты.
 - Скрипты для тренировки, тестирования, вычисления метрик и инференса моделей.
 - Конфигурационные файлы для запуска моделей.
4. Средства контейнеризации Docker и скрипты для автоматической установки зависимостей.
5. RTSP потоки заказчика для проведения демонстрационного инференса или подготовленные видеоролики.
6. Установленная операционная система Ubuntu версии 22+ и JetPack 5.
7. Техническое задание с требованиями к форматам выходных данных.

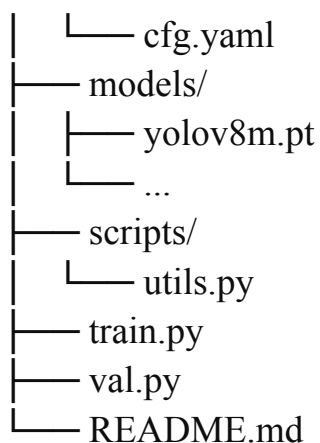
Проверить окружение:

- **Входные данные:** RTSP потоки, архивные видеоданные, статические изображения.
- **Выходные данные:**
 - JSON файлы с результатами детекции в формате timestamp.json.
 - Видеоролики с наложением детектированных объектов в формате timestamp.mp4.

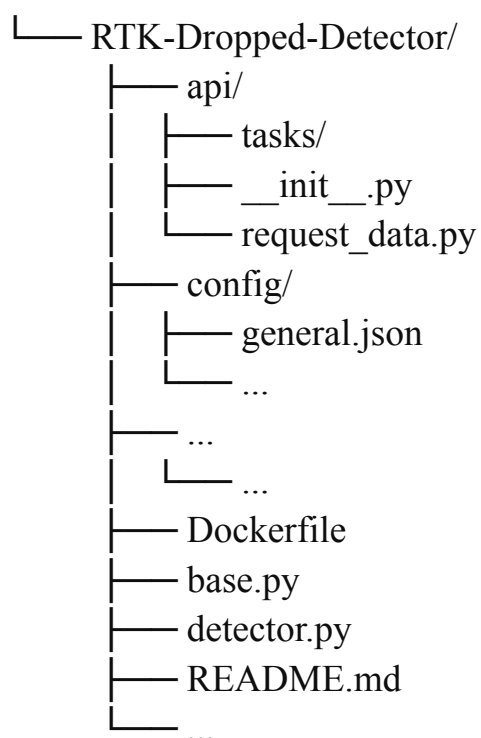
Структура репозитория с датасетом и обучающими скриптами выглядит следующим образом (пример: детектор марок/моделей):

```

└─ make_model_detector/
    │
    └─ dataset/
        │
        └─ train
        └─ val
  
```



Структура репозитория для подключения к платформе AP выглядит следующим образом (пример: детектор выпавшего груза):



Приоритетными считать инструкции приложенные к исходному коду проекта. Последующие шаги предоставляются в ознакомительных целях.

Начать испытания согласно Сценарию:

Шаг 1. Подготовка репозитория

1. Развернуть Docker-контейнер для соответствующего детектора с использованием предоставленного скрипта для работы с платформой AP:


```
docker build -t detector_image ./
docker run --rm -it --name base-detector-api-container-p 8000:8000 -v
./logs:/detector-api/logs/ -v ./videos:/detector-api/videos/
detector_image
```

2. Проверить наличие всех необходимых файлов в репозитории:

- Датасетов (тренировочных, валидационных, тестовых).
- Скриптов для тренировки, тестирования и инференса.
- Конфигурационных файлов.

Ожидаемый результат: Все файлы успешно доступны, контейнер запускается и не выдаёт ошибок.

Шаг 2. Тренировка модели

1. Запустить скрипт тренировки модели с указанием тренировочного датасета:

```
python train.py --dataset training_dataset
```

2. Убедиться, что модель сохраняет контрольные точки и генерирует логи метрик (точность, полнота и AP).

Ожидаемый результат: Модель успешно обучена, контрольные точки сохранены, метрики вычислены.

Шаг 3. Тестирование модели на валидационном датасете

1. Выполнить скрипт для вычисления метрик на валидационном датасете:

```
python val.py --dataset validation_dataset
```

2. Сравнить метрики модели с минимальными значениями, указанными в ТЗ:

- Точность (Precision).
- Полнота (Recall).
- Среднее значение точности (AP) на порогах 0.5 и 0.95.

Ожидаемый результат: Метрики соответствуют минимальным значениям ТЗ.

Шаг 4. Демонстрационный инференс на RTSP потоке

1. Запустить инференс на RTSP потоке с ограничением по времени (15 секунд):

```
python infer.py --stream rtsp://example_stream --duration 15
```

2. Убедиться, что результаты инференса сохранены:

- JSON-файл `timestamp.json` с информацией о детектированных объектах.
- Видеофайл `timestamp.mp4` с наложением объектов и ключевых параметров.

Ожидаемый результат:

- JSON-файл включает информацию о детектированных объектах, их координатах и классах.
- Видеофайл визуализирует детектированные объекты с наложением данных.

Шаг 5. Проверка интеграции

1. Для моделей с подключением к веб-интерфейсу:

- Убедиться, что данные передаются корректно и отображаются в указанных интерфейсах.

2. Для остальных моделей:

- Проверить сохранение результатов в папке `results`.

Ожидаемый результат: Результаты корректно передаются в интерфейсы или сохраняются в указанной директории.

Общий итоговый Ожидаемый результат

1. Модель корректно обучена и протестирована, результаты соответствуют минимальным требованиям ТЗ.
2. JSON-файлы и видеоролики генерируются в указанном формате.
3. Интеграция с Телеграм-ботом, веб-интерфейсом или файловой системой функционирует без ошибок.
4. Система готова к демонстрации и интеграции в целевую среду.

Смотри таблицу для отчета об испытаниях в следующем разделе.

7. Результаты предварительных испытаний

По результатам разработки для каждой из моделей детектирования вычислены и предоставлены следующие количественные параметры (метрики) модели:

Работоспособность детектора на собственных видеопотоках или фрагментах видео								
	Ожидаемый результат				Фактический результат теста			
	специфичность	точность	AP (0.5)	AP (0.95)	полнота	точность	AP (0.5)	AP (0.95)
Детектор марок, моделей и цвета ТС	0.7	0.7	0.6	-	0.75	0.75	-	-
Детектор ДТП на перекрестках	0.86	0.9	-	0.37	0.9	0.9	-	0.5
Детектор выпавшего груза на перекрестке	0.8	0.8	0.5	-	0.92	0.55	0.55	-
Детектор остановки в полосе движения	0.87	0.83	-	0.5	0.92	0.9	-	-
Детектор пешехода	0.87	0.83	-	0.5	0.83	0.91	0.91	0.59
Детектор непристегнутого ремня / разговора по телефону	0.8	0.8	0.5	-	0.8	0.76	0.61	-

Количественные показатели предоставляемых детекторов соответствуют заявленным уровням Технического задания.

8. Сценарии для испытаний детекторов

Рассмотрим сценарии испытаний следующих детекторов:

- Детектор марки, модели и цвета ТС
- Детектор ДТП на перекрестке
- Детектор выпавшего груза на перекрестке
- Детектор остановки в полосе движения
- Детектор пешехода
- Детектор непристегнутого ремня / разговора по телефону

Приоритетными следует считать инструкции, приложенные к исходному коду проекта каждого детектора. Последующие сценарии предоставляются в ознакомительных целях.

Эти сценарии позволяют провести детальное тестирование каждого детектора с учётом их специфики, обеспечивая полное соответствие функциональности системы заявленным требованиям.

Сценарий 1: Тестирование детектора марки, модели и цвета транспортного средства

Предусловия

1. Доступ к репозиторию модели с исходными данными.
2. Настроенный Docker-контейнер с установленными зависимостями.
3. Витопоток или изображения с транспортными средствами различных марок, моделей и цветов.

Окружение

- **Аппаратные требования:**

- Сервер виртуальных машин ВКИ с конфигурацией 8 ядер CPU, 16Гб ОЗУ или одноплатный компьютер NVIDIA Jetson NX Xavier 8Gb.
- **Камеры:**
 - Разрешение не менее **1920x1080** для улиц.
 - Камеры должны быть установлены так, чтобы исключить засветки и попадание прямых солнечных лучей на объектив.
- **Программные требования:**
 - Операционная система: **РЕД ОС 7.3, Ubuntu 22.04+** или **JetPack 5**.
 - Установленный Docker.
- **Сетевые требования:**
 - Стабильное подключение с задержкой не более 5 минут между детекцией и передачей данных.
- **Входные данные:**
 - RTSP-видеопоток или архив изображений, обеспечивающий разнообразие сцен и условий.

Сценарий тестирования

1. Подготовка оборудования:

- Убедиться, что аппаратные требования соответствуют указанным для обработки видеопотока.
- Проверить корректность установки камер и настройки их разрешения (1920x1080 для улиц).

2. Запуск Docker-контейнера:

```
docker run -p 8000:8000 -v ./logs:/detector-api/logs/ -it  
detector_vehicle_model
```

3. Запуск инференса на видеопотоке и его анализ:

```
python infer.py --stream rtsp://example_stream --duration 30
```

4. Проверка JSON-выходных данных:

- Убедиться, что вывод содержит три поля: “марка”, “модель”, “цвет”.

5. Валидация результатов:

- Сравнить выданные результаты с фактическими данными.

Ожидаемый результат

- JSON-файл содержит точную информацию о марке, модели и цвете транспортного средства.
- Формат данных:

```
{
  "timestamp": 1674487988888,
  "request_counter": 22,
  "message_type": 1,
  "confidence": 0.723,
  "color": "RED",
  "vehicle_type": 6,
  "brand": "Hyundai",
  "model": "Solaris"
}
```

Сценарий 2: Тестирование детектора ДТП на перекрестках

Предусловия

1. Видеопоток с событиями на перекрестке, включая ДТП и движения без происшествий.
2. Модель детектора с активированными фильтрами для обнаружения пересечения bounding boxes.
 - **Аппаратные требования:**
 - Сервер виртуальных машин ВКИ с конфигурацией 8 ядер CPU, 16Гб ОЗУ.
 - **Камеры:**
 - Разрешение не менее **1920x1080** для улиц.
 - Камеры должны быть установлены так, чтобы исключить засветки и попадание прямых солнечных лучей на объектив.
 - **Программные требования:**
 - Операционная система: **РЕД ОС 7.3** или **Ubuntu 22.04+**.
 - Установленный Docker.
 - **Сетевые требования:**
 - Стабильное подключение с задержкой не более 5 минут между детекцией и передачей данных.
 - **Входные данные:**
 - RTSP-видеопоток или архив изображений, обеспечивающий разнообразие сцен и условий.

Сценарий тестирования

1. Подготовка оборудования:

- Убедиться, что аппаратные требования соответствуют указанным для обработки видеопотока.
- Проверить корректность установки камер и настройки их разрешения (1920x1080 для улиц).

2. Запуск Docker-контейнера:

```
docker run -p 8000:8000 -v ./logs:/detector-api/logs/ -it detector_carcrash
```

3. Запуск инференса на видеопотоке и его анализ:

```
python infer.py --stream rtsp://example_stream --duration 30
```

4. Проверка метрик пересечения bounding boxes:

- Убедиться, что пересечения автомобилей фиксируются корректно.

5. Анализ выхода нейронной сети:

- Подтверждение ДТП при резком снижении скорости или наличии пересечения bounding boxes.

Ожидаемый результат

- JSON-выход:

```
{
  "event": "accident",
  "vehicle1_id": 5,
  "vehicle2_id": 10,
  "timestamp": 1674487988888,
  "box": {
    "left": 0.634375,
    "right": 0.7054688,
    "top": 0.40555555,
    "bottom": 0.44444445
  },
  "accuracy": 0.92
}
```

- Видеофайл с визуализацией ДТП (подпись на видео).

Сценарий 3: Тестирование детектора остановок на полосе движения

Предусловия

1. Набор видеоданных с остановками транспортных средств.
2. Репозиторий детектора с готовыми скриптами для тестирования.

Окружение

- **Аппаратные требования:**
 - Сервер виртуальных машин ВКИ с конфигурацией 8 ядер CPU, 16 Гб ОЗУ.
- **Камеры:**
 - Разрешение не менее **1920x1080** для улиц.
 - Камеры должны быть установлены так, чтобы исключить засветки и попадание прямых солнечных лучей на объектив.
- **Программные требования:**
 - Операционная система: **РЕД ОС 7.3** или **Ubuntu 22.04+**.
 - Установленный Docker.
- **Сетевые требования:**
 - Стабильное подключение с задержкой не более 5 минут между детекцией и передачей данных.
- **Входные данные:**
 - RTSP-видеопоток или архив изображений, обеспечивающий разнообразие сцен и условий.

Сценарий тестирования

1. **Подготовка оборудования:**
 - Убедиться, что аппаратные требования соответствуют указанным для обработки видеопотока.
 - Проверить корректность установки камер и настройки их разрешения (1920x1080 для улиц).
2. **Запуск Docker-контейнера:**

```
docker run -p 8000:8000 -v ./logs:/detector-api/logs/ -it  
detector_lane_stop
```

3. **Запуск инференса на видеопотоке и его анализ:**


```
python infer.py --stream rtsp://example_stream --duration 30
```

4. Проверка времени остановок:

- Убедиться, что фиксируются остановки объектов при скорости ниже среднего значения для сектора в течение 30 секунд.

5. Сравнение результата с эталонными данными:

- Сравнить выход модели с фактическими остановками.

Ожидаемый результат

- JSON-выход:

```
{
  "event": "stop",
  "vehicle_id": 7,
  "timestamp": 1674487988888,
  "box": {
    "left": 0.634375,
    "right": 0.7054688,
    "top": 0.40555555,
    "bottom": 0.44444445
  },
  "duration": 60
}
```

- Видеофайл с визуализацией остановки объекта (выделение красной рамкой).

Сценарий 4: Тестирование детектора выпавшего груза на перекрестке

Предусловия

1. Доступ к видеопотокам перекрестков с движением грузового транспорта, включающим события выпадения груза.
2. Репозиторий модели детектора для распознавания объектов, находящихся продолжительное время без движения.
3. Подготовленный RTSP поток для инференса.

Окружение

- **Аппаратные требования:**
 - Сервер виртуальных машин ВКИ с конфигурацией 8 ядер CPU, 16Гб ОЗУ.

- **Камеры:**
 - Разрешение не менее **1920x1080** для улиц.
 - Камеры должны быть установлены так, чтобы исключить засветки и попадание прямых солнечных лучей на объектив.
- **Программные требования:**
 - Операционная система: **РЕД ОС 7.3** или **Ubuntu 22.04+**.
 - Установленный Docker.
- **Сетевые требования:**
 - Стабильное подключение с задержкой не более 5 минут между детекцией и передачей данных.
- **Входные данные:**
 - RTSP-видеопоток или архив изображений, обеспечивающий разнообразие сцен и условий.

Сценарий тестирования

1. Подготовка оборудования:

- Убедиться, что аппаратные требования соответствуют указанным для обработки видеопотока.
- Проверить корректность установки камер и настройки их разрешения (1920x1080 для улиц).

2. Запуск Docker-контейнера:

```
docker run -p 8000:8000 -v ./logs:/detector-api/logs/ -it detector_dropped
```

3. Запуск инференса на видеопотоке и его анализ:

```
python infer.py --stream rtsp://example_stream --duration 30
```

4. Проверка идентификации выпавших объектов:

- Убедиться, что фиксируются события отделения груза от транспортного средства.
- Проверить, что модель различает выпавший груз от посторонних объектов на дороге (например, мусора).

5. Анализ результата инференса:

- Проверить JSON-выходной файл на соответствие указанным в ТЗ требованиям.

Ожидаемый результат

- JSON-выход:

```
{
  "event": "dropped",
  "object_id": 56,
  "box": {
    "left": 0.634375,
    "right": 0.7054688,
    "top": 0.40555555,
    "bottom": 0.44444445
  },
  "time_detected": 60
}
```

- Видеофайл с визуализацией объекта, выделенного в момент отделения.

Сценарий 5: Тестирование детектора пешехода

Предусловия

1. Наличие видеопотока с пешеходами на пешеходных переходах и вне их.
2. Репозиторий детектора с обученной моделью для распознавания человеческих силуэтов.

Окружение

- **Аппаратные требования:**
 - Сервер виртуальных машин ВКИ с конфигурацией 8 ядер CPU, 16Гб ОЗУ.
- **Камеры:**
 - Разрешение не менее **1920x1080** для улиц.
 - Камеры должны быть установлены так, чтобы исключить засветки и попадание прямых солнечных лучей на объектив.
- **Программные требования:**
 - Операционная система: **РЕД ОС 7.3** или **Ubuntu 22.04+**.
 - Установленный Docker.
- **Сетевые требования:**

- Стабильное подключение с задержкой не более 5 минут между детекцией и передачей данных.
- **Входные данные:**
 - RTSP-видеопоток или архив изображений, обеспечивающий разнообразие сцен и условий.

Сценарий тестирования

1. Подготовка оборудования:

- Убедиться, что аппаратные требования соответствуют указанным для обработки видеопотока.
- Проверить корректность установки камер и настройки их разрешения (1920x1080 для улиц).

2. Запуск Docker-контейнера:

```
docker run -p 8000:8000 -v ./logs:/detector-api/logs/ -it  
detector_pedestrian
```

3. Запуск инференса на видеопотоке и его анализ:

```
python infer.py --stream rtsp://example_stream --duration 30
```

4. Идентификация пешеходов:

- Убедиться, что пешеходы корректно выделяются в кадре, включая тех, кто находится на переходе и вне зоны перехода.
- Проверить, что система различает пешеходов при различных ракурсах и масштабах и не локализует другие объекты (например, животных).

5. Анализ результата:

- Проверить JSON-выходной файл и видеоролик с визуализацией.

Ожидаемый результат

- JSON-выход:

```
{  
  "event": "pedestrian",  
  "pedestrian_id": 54,  
  "box": {
```

```

    "left": 0.634375,
    "right": 0.7054688,
    "top": 0.40555555,
    "bottom": 0.44444445
  },
  "accuracy": 0.95
}

```

- Видеофайл с выделением пешеходов и их зон.

Сценарий 6: Тестирование детектора непристегнутого ремня и разговора по телефону

Предусловия

1. Видеозаписи с крупным планом водителей, включая сценарии:
 - Водитель пристегнут / не пристегнут.
 - Водитель использует телефон / не использует телефон.
2. Репозиторий модели с готовыми скриптами для анализа состояния водителя.

Окружение

- **Аппаратные требования:**
 - Сервер виртуальных машин ВКИ с конфигурацией 8 ядер CPU, 16Гб ОЗУ.
- **Камеры:**
 - Разрешение не менее **1920x1080** для улиц.
 - Камеры должны быть установлены так, чтобы исключить засветки и попадание прямых солнечных лучей на объектив.
- **Программные требования:**
 - Операционная система: **РЕД ОС 7.3** или **Ubuntu 22.04+**.
 - Установленный Docker.
- **Сетевые требования:**
 - Стабильное подключение с задержкой не более 5 минут между детекцией и передачей данных.
- **Входные данные:**
 - RTSP-видеопоток или архив изображений, обеспечивающий разнообразие сцен и условий.

Сценарий тестирования

1. Подготовка оборудования:

- Убедиться, что аппаратные требования соответствуют указанным для обработки видеопотока.
- Проверить корректность установки камер и настройки их разрешения (1920x1080 для улиц).

2. Запуск Docker-контейнера:

```
docker run -p 8000:8000 -v ./logs:/detector-api/logs/ -it  
detector_seatbelt_phone
```

3. Запуск инференса на видеопотоке и его анализ:

```
python infer.py --stream rtsp://example_stream --duration 30
```

4. Проверка вывода:

- Проверить идентификацию состояния ремня безопасности.
- Убедиться, что система фиксирует использование телефона.

5. Анализ результата инференса:

- Сравнить данные инференса с фактическими условиями.

Ожидаемый результат

- JSON-выход:

```
{  
  "event": "driver_violation",  
  "no_seatbelt": false,  
  "phone": true,  
  "accuracy": 0.91  
}
```

- Набор изображений с выделением нарушений и наложением пояснений.

